

Projektübersicht

Projektname

Steam Library REST-API (M295)

Projektidee

Die Applikation „Steam Library REST-API“ ermöglicht die Verwaltung einer persönlichen Spielebibliothek.

Benutzer können Spiele erfassen, bearbeiten, löschen und filtern.

Alle Daten werden persistent in einer relationalen MySQL-Datenbank gespeichert.

Der Fokus des Projektes liegt auf einer sauberen REST-Architektur mit Spring Boot, serverseitiger Validierung, Fehlerbehandlung und Unit-Tests.

Ziel des Projektes

Am Ende des Projektes soll:

- eine vollständig lauffähige REST-API vorhanden sein
- eine relationale Datenbank verwendet werden
- CRUD-Operationen über REST bereitgestellt werden
- Validierung und Fehlerbehandlung implementiert sein
- eine vollständige Projektdokumentation vorliegen

Anforderungen

Funktionale Anforderungen

- Spiele müssen persistent gespeichert werden.
- CRUD-Operationen müssen über REST verfügbar sein.
- Daten müssen serverseitig validiert werden.
- Fehler müssen als strukturierte HTTP-Responses zurückgegeben werden.
- Spiele müssen nach bestimmten Kriterien filterbar sein.

Nicht-Ziel

Ein Frontend ist nicht Bestandteil der Bewertung im Modul.

User Stories

User Story 1 – Spiel erfassen

Als Benutzer möchte ich ein neues Spiel erfassen, damit ich meine Bibliothek erweitern kann.

****Akzeptanzkriterien:****

- POST /games erstellt ein neues Spiel
- Pflichtfelder dürfen nicht leer sein
- HTTP 201 wird zurückgegeben

User Story 2 – Spiel anzeigen

Als Benutzer möchte ich alle Spiele anzeigen können, damit ich eine Übersicht meiner Bibliothek habe.

****Akzeptanzkriterien:****

- GET /games liefert alle Spiele
- HTTP 200 wird zurückgegeben

User Story 3 – Spiel bearbeiten

Als Benutzer möchte ich ein Spiel aktualisieren können, damit ich Spielzeit oder Status anpassen kann.

****Akzeptanzkriterien:****

- PUT /games/{id} aktualisiert das Spiel
- Bei nicht vorhandener ID → HTTP 404

User Story 4 – Spiel löschen

Als Benutzer möchte ich ein Spiel löschen können, um meine Bibliothek zu bereinigen.

****Akzeptanzkriterien:****

- DELETE /games/{id} löscht das Spiel
- HTTP 204 bei Erfolg

Architektur

Technologie-Stack

Die Applikation wurde mit folgenden Technologien umgesetzt:

- Java 17
- Spring Boot 3
- Maven
- MySQL
- JPA / Hibernate
- JUnit (Unit-Tests)

Systemarchitektur

Die Architektur folgt dem klassischen Schichtenmodell:

Client → Controller → Repository → Datenbank

1. Ein Client sendet einen HTTP-Request an die REST-API.
2. Der Controller verarbeitet die Anfrage.
3. Die Daten werden mittels Bean Validation validiert.
4. Das Repository greift über JPA auf die MySQL-Datenbank zu.
5. Eine strukturierte JSON-Response wird zurückgegeben.

Klassendiagramm

Klasse: Game

Attribut	Typ	Beschreibung
id	Long	Primärschlüssel
steamAppId	Integer	Eindeutige Steam-ID
title	String	Name des Spiels
playtimeHours	Integer	Spielzeit in Stunden
installed	Boolean	Installationsstatus
lastPlayed	LocalDate	Datum der letzten Nutzung

Aktuell besteht das System aus einer zentralen Entität (`Game`).

Beziehungen zu weiteren Entitäten sind nicht implementiert.

Architektur

Technologie-Stack

Die Applikation wurde mit folgenden Technologien umgesetzt:

- Java 17
- Spring Boot 3
- Maven
- MySQL
- JPA / Hibernate
- JUnit (Unit-Tests)

Systemarchitektur

Die Architektur folgt dem klassischen Schichtenmodell:

Client → Controller → Repository → Datenbank

1. Ein Client sendet einen HTTP-Request an die REST-API.
2. Der Controller verarbeitet die Anfrage.
3. Die Daten werden mittels Bean Validation validiert.
4. Das Repository greift über JPA auf die MySQL-Datenbank zu.
5. Eine strukturierte JSON-Response wird zurückgegeben.

Klassendiagramm

Klasse: Game

| Attribut | Typ | Beschreibung |

|-----|-----|-----|

| id | Long | Primärschlüssel |

| steamAppld | Integer | Eindeutige Steam-ID |

| title | String | Name des Spiels |

| playtimeHours | Integer | Spielzeit in Stunden |

| installed | Boolean | Installationsstatus |

| lastPlayed | LocalDate | Datum der letzten Nutzung |

Aktuell besteht das System aus einer zentralen Entität (`Game`).

Beziehungen zu weiteren Entitäten sind nicht implementiert.

Testing

Unit-Tests

Zur Sicherstellung der Kernfunktionalität wurden 5 Unit-Tests implementiert.

Getestet werden:

- Abrufen aller Spiele
- Erstellen eines gültigen Spiels
- Erstellen eines ungültigen Spiels
- Löschen eines Spiels
- Fehlerfall bei nicht existierender ID

Die Tests können mit folgendem Befehl ausgeführt werden:

```
mvn test
```

Manueller Testplan

Testfall Erwartetes Ergebnis
----- -----
Neues Spiel erstellen HTTP 201
Spiel ohne Pflichtfeld HTTP 400
Spiel abrufen HTTP 200
Spiel löschen HTTP 204
Ungültige ID abrufen HTTP 404

Testdurchführung

Alle definierten Testfälle wurden erfolgreich durchgeführt.

Die API reagiert stabil und liefert korrekte HTTP-Statuscodes.

Betrieb & Installation

Voraussetzungen

- Java 17

- Maven

- MySQL

Datenbank Setup

1. MySQL starten

2. Neue Datenbank erstellen:

CREATE DATABASE steam_library;

3. Optional: Benutzer steam_user erstellen

Backend starten

Im Backend-Ordner:

mvn spring-boot:run

Die API ist anschliessend erreichbar unter:

<http://localhost:8080>

Projekt in IDE importieren

1. Projekt als Maven-Projekt öffnen

2. Dependencies automatisch laden lassen

3. Projekt starten

Reflexion

Im Rahmen dieses Projektes konnte ich mein Wissen im Bereich Backend-Entwicklung vertiefen.

Besonders konnte ich folgende Themen praktisch anwenden:

- Aufbau einer REST-API mit Spring Boot
- Persistente Speicherung mit JPA und MySQL
- Serverseitige Validierung von Daten
- Strukturierte Fehlerbehandlung
- Erstellung von Unit-Tests
- Dokumentation eines Softwareprojektes

Eine Herausforderung war die saubere Umsetzung der REST-Endpunkte sowie die vollständige und strukturierte Projektdokumentation.

Das Projekt erfüllt die Anforderungen des Moduls M295 und ist vollständig lauffähig.